TensorForge 2.1.0 — Design, evaluation and findings

Model: `tf-phase2-drive-2.1.0` · **Release:** `v2.1.0` · **Product:** Fresh Desk support-ticket routing for RideEat

Fresh Desk reads a customer support ticket (chat, email, or call transcript, in English, Sinhala, Tamil, their Latin-script forms, or a mix) and predicts three things: the **primary category** (11 classes), an optional **secondary category**, and whether the ticket is **urgent**. The support **team** is not predicted; it is derived from the primary category with the fixed routing table in the contract.

This report has three parts:

1. **Design decisions** — what we built and why, including what we rejected.
2. **Evaluation scores** — held-out results on a deterministic, stratified 75/25 split of `train.csv` (3,000 fit / 1,000 untouched test tickets).
3. **Findings** — what the scores show, where the model is weak, and the limits of this evaluation.

All scores come from models fitted on the 75% fold. The deployed artifact was then refit on all 4,800 labelled rows (`train.csv` plus `validation.csv`) and must not be re-scored on those rows. The data are synthetic workshop examples: good scores here do not guarantee quality on real customer messages or on the judges' independent data.

Setup

Scores are read from `artifacts/evaluation_metrics.json` . Set `RETRAIN = True` to reproduce the split, the held-out scoring, and the final refit with `scripts/train_model.py` . Retraining **overwrites** `artifacts/model.joblib` and the saved metrics.

```
tf-phase2-drive-2.1.0
Metrics are from the untouched 25% stratified test fold of train.csv. After scor
ing, the deployment artifact was refit on train.csv plus validation.csv; do not
evaluate that final artifact on these same examples.
```

Part 1 — Design decisions

Area	Decision	Why	Rejected or avoided
Approach	Our own trained classifiers, served locally	The brief discourages LLM wrappers; local models need no network, key, or download at runtime	Calling an LLM or a hosted model API
Features	Word 1–2 grams + character 2–6 grams (TF-IDF)	Words capture issue phrases; character fragments survive misspellings, Sinhala/Tamil scripts, and transliteration — no translation step	Translating to English first; ticket IDs and dataset language labels as features
Targets	Three separate models, one per output	Each output has different class balance and needs its own threshold	One joint model for all outputs
Team	Derived from the primary category via the fixed table	The contract defines the mapping; predicting it separately could contradict the category	A fourth "team" classifier
Consistency	Post-prediction rules: secondary ≠ primary; spam is never urgent and has no secondary	Guarantees contract-valid output even when a model disagrees	Trusting raw model outputs
Human review	Recommended when primary confidence < 0.5 or the message has ≤ 3 words; urgent tickets go to the support queue	Keeps a person in the loop exactly where the model is least reliable	Fully automatic routing
Model selection	Configurations and cutoffs chosen on the supplied development validation set	A first candidate chosen on an internal training split scored poorly on validation and was dropped (see Findings)	Selecting on internal splits of synthetic data
Serving	FastAPI; auth → content type → size → JSON → validation, in contract order; sync batches ≤ 100, async jobs ≤ 5,000	An unauthorised or malformed request never reaches the model; large runs don't block health checks	Unbounded synchronous batches
Packaging	One Docker image with UI, model, and dependencies; runs with networking disabled	Matches the evaluation's offline, single- `docker run` requirement	Downloading the model at start-up

1.1 Models and why

All targets share a scikit-learn TF-IDF pipeline. Ticket channel, subject, and message are features; an optional language hint is used when supplied. Text is Unicode NFC-normalized; secondary and urgency preprocessing also normalizes spacing, masks synthetic references, and removes explicitly quoted past correspondence.

- **Primary category — balanced Logistic Regression, `C=100`.** Efficient on sparse word and boundary-aware `char_wb` 2–6-gram features, and returns class probabilities for routing and confidence. Balanced class weights reduce majority-class preference.
- **Secondary category — sigmoid-calibrated Linear SVM, `C=1`, cutoff `0.20`.** A margin classifier suits sparse high-dimensional text; three-fold sigmoid calibration gives probabilities for the optional-label cutoff. The low cutoff reflects that a second issue is optional and must differ from the primary.
- **Urgency — balanced Logistic Regression, `C=40`, cutoff `0.30`.** Probabilities support an explicit operating threshold, set below 0.5 to favour catching urgent tickets; class balancing keeps the rarer urgent class from being ignored.

The frozen configuration from `configs/training.json`:

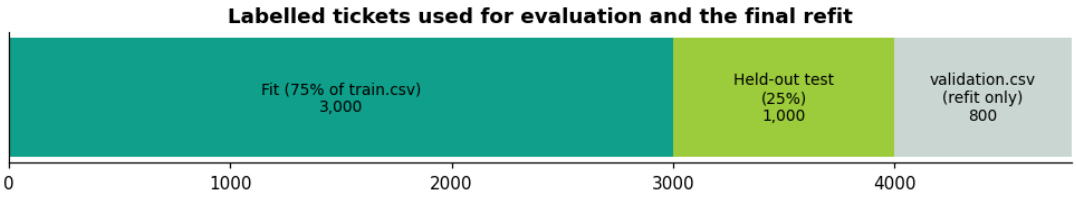
target	estimator	C	class weight	normalize	char analyzer	char n-grams	decision threshold
Primary category	logistic_regression	100.0	balanced	False	char_wb	[2, 6]	argmax
Secondary category	calibrated_svm	1.0	None	True	char	[2, 6]	0.2
Urgency	logistic_regression	40.0	balanced	True	char	[2, 5]	0.3

Part 2 — Evaluation scores

2.1 Data and split

Only the organisers' synthetic workshop data is used. The split is stratified by primary category with a fixed seed (42), so it is reproducible. `validation.csv` is not used for test scoring; it joins the training data only for the final refit.

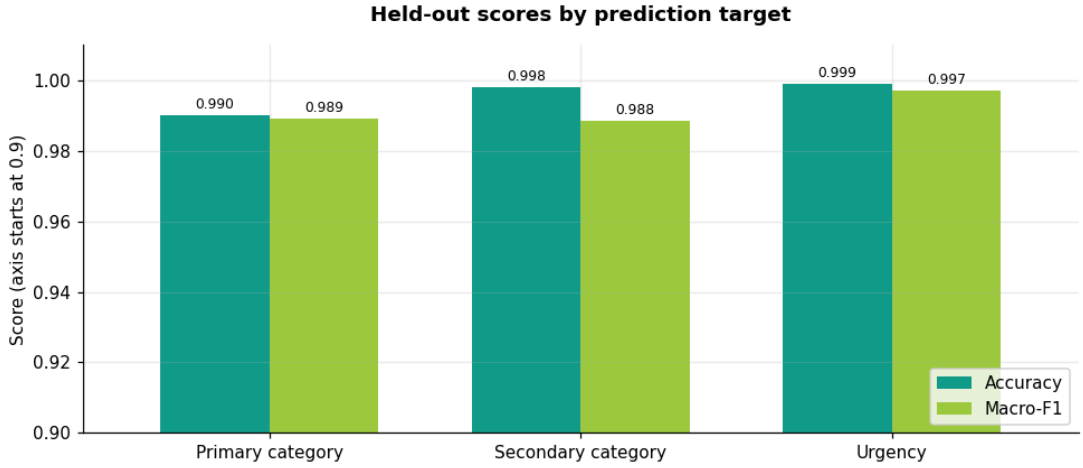
	value
method	stratified random split by primary category from train.csv only
test_size	0.25
random_state	42
fit_examples	3000
test_examples	1000
validation_file_used_for_test_scoring	False



2.2 Overall held-out scores

Accuracy is the fraction of correct predictions. Macro-F1 averages per-class F1 with every class weighted equally, so rare labels count as much as common ones. Secondary labels include a `__none__` outcome. Scores include the consistency and precedence rules.

prediction target	accuracy (%)	macro-F1 (%)
Primary category	99.0	98.92
Secondary category	99.8	98.84
Urgency	99.9	99.71



2.3 Every class: precision, recall, F1, and support

Support is the number of test tickets for the class. Sparse secondary labels have low support, so their scores are less certain.

Primary category

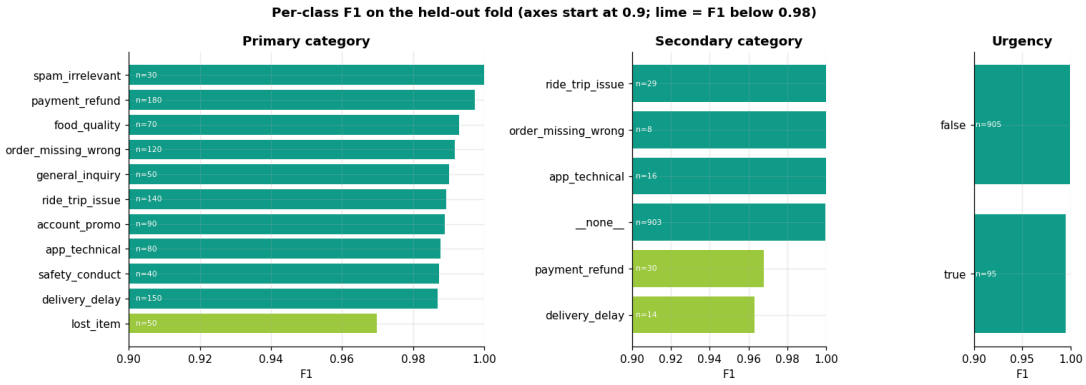
	precision	recall	f1-score	support
account_promo	0.9889	0.9889	0.9889	90
app_technical	0.9756	1.0000	0.9877	80
delivery_delay	0.9740	1.0000	0.9868	150
food_quality	1.0000	0.9857	0.9928	70
general_inquiry	0.9804	1.0000	0.9901	50
lost_item	0.9796	0.9600	0.9697	50
order_missing_wrong	1.0000	0.9833	0.9916	120
payment_refund	1.0000	0.9944	0.9972	180
ride_trip_issue	0.9928	0.9857	0.9892	140
safety_conduct	1.0000	0.9750	0.9873	40
spam_irrelevant	1.0000	1.0000	1.0000	30
macro avg	0.9901	0.9885	0.9892	1000
weighted avg	0.9901	0.9900	0.9900	1000

Secondary category

	precision	recall	f1-score	support
__none__	1.0000	0.9989	0.9994	903
app_technical	1.0000	1.0000	1.0000	16
delivery_delay	1.0000	0.9286	0.9630	14
order_missing_wrong	1.0000	1.0000	1.0000	8
payment_refund	0.9375	1.0000	0.9677	30
ride_trip_issue	1.0000	1.0000	1.0000	29
macro avg	0.9896	0.9879	0.9884	1000
weighted avg	0.9981	0.9980	0.9980	1000

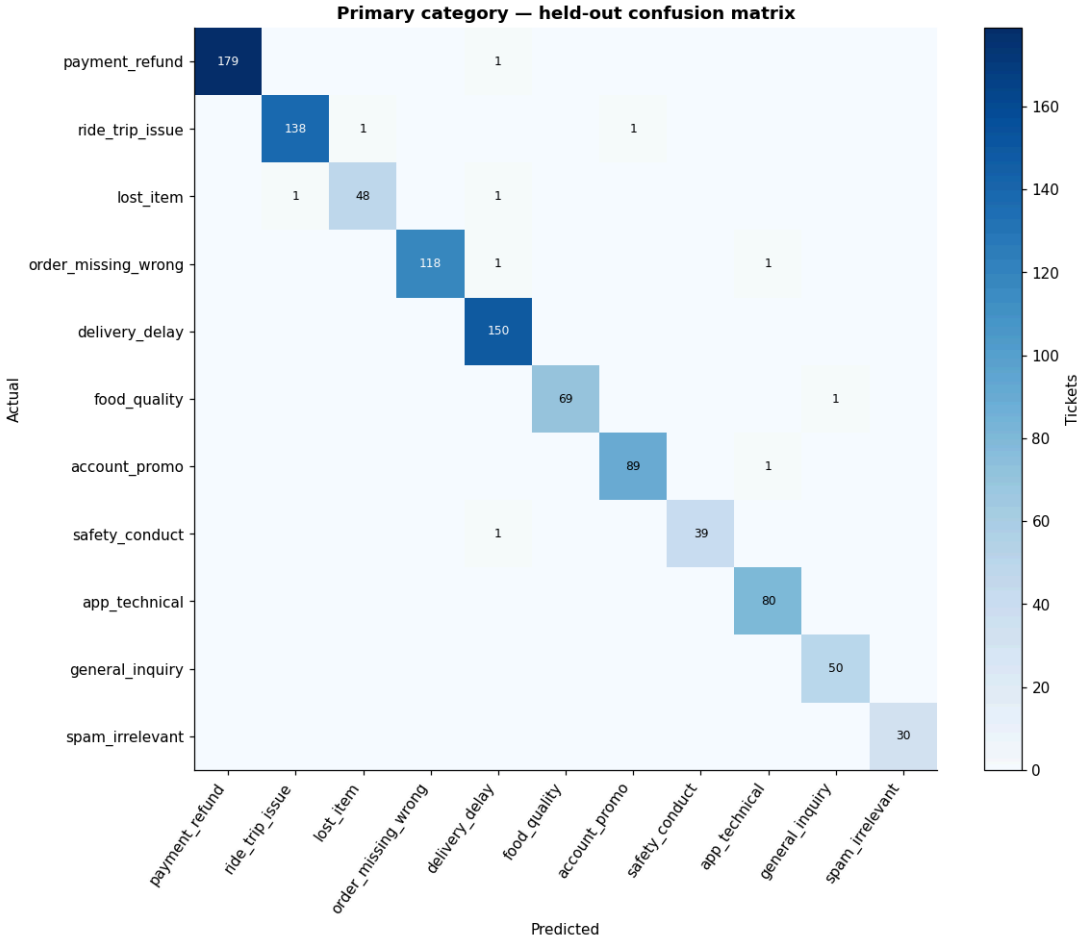
Urgency

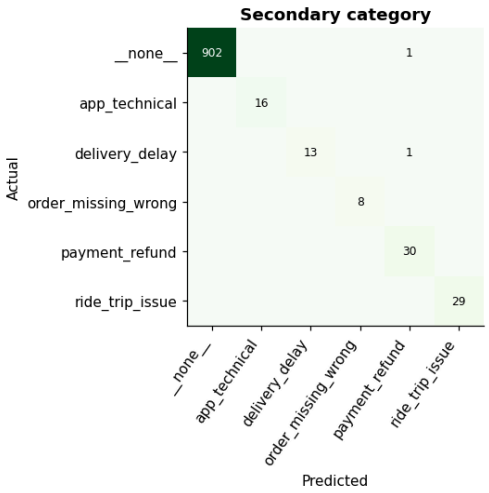
	precision	recall	f1-score	support
false	0.9989	1.0000	0.9994	905
true	1.0000	0.9895	0.9947	95
macro avg	0.9994	0.9947	0.9971	1000
weighted avg	0.9990	0.9990	0.9990	1000

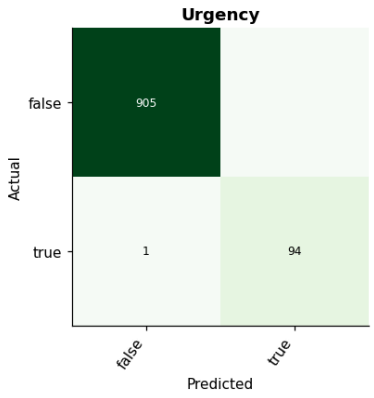


2.4 Confusion matrices

Rows are actual labels; columns are predicted labels, after the precedence and consistency rules.







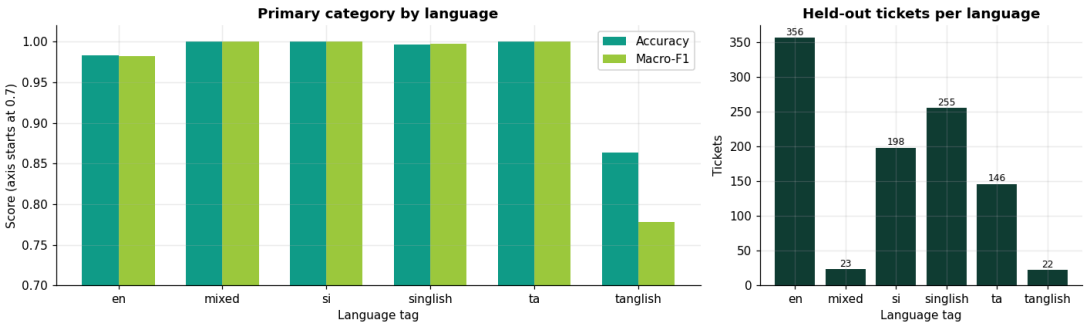
Most frequent primary-category mistakes:

	actual	predicted	count
0	order_missing_wrong	delivery_delay	1
1	ride_trip_issue	account_promo	1
2	ride_trip_issue	lost_item	1
3	lost_item	ride_trip_issue	1
4	safety_conduct	delivery_delay	1
5	order_missing_wrong	app_technical	1
6	lost_item	delivery_delay	1
7	account_promo	app_technical	1
8	food_quality	general_inquiry	1
9	payment_refund	delivery_delay	1

2.5 Performance by language

Primary-category scores grouped by the dataset language tag. Group sizes differ greatly; the `mixed` and `tanglish` groups have only about twenty test tickets each.

	count	accuracy	macro_f1
en	356	0.9831	0.9825
mixed	23	1.0000	1.0000
si	198	1.0000	1.0000
singlish	255	0.9961	0.9978
ta	146	1.0000	1.0000
tanglish	22	0.8636	0.7778



2.6 Delivery-delay vs. lost-property routing

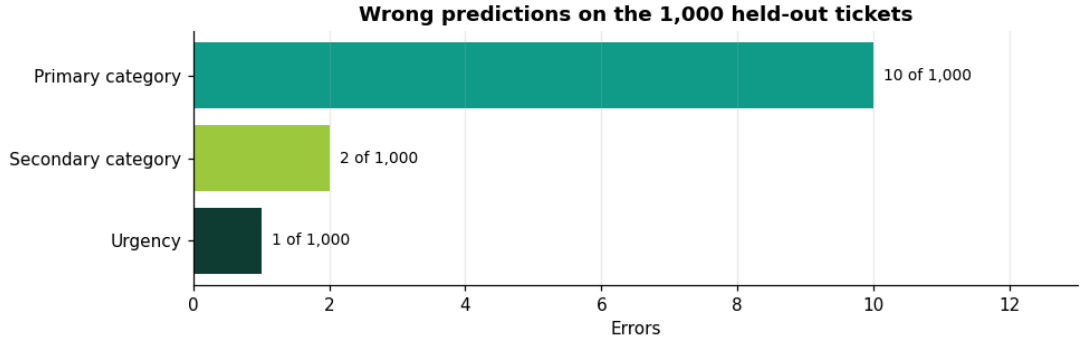
A delayed, not-yet-arrived delivery is `delivery_delay`; belongings left during a ride are `lost_item`. The first table isolates these two categories from the held-out confusion matrix. The live predictions are a smoke check of the deployed full-data artifact, not part of the held-out score.

	predicted lost_item	predicted delivery_delay
actual lost_item	48	1
actual delivery_delay	0	150

	message	category	team	confidence
0	My food order is delayed and still has not arrived.	delivery_delay	Delivery Operations	0.9977
1	I left my phone in the car after my ride.	lost_item	Lost & Found	0.9998

Part 3 — Findings

The findings below are computed from the saved metrics, so they always match the scores in Part 2.



What works

1. **All three outputs score highly on the held-out fold.** Macro-F1 is 98.92% for the primary category, 98.84% for the secondary category, and 99.71% for urgency. In total there are 13 wrong predictions across 3,000 predictions.
2. **One small model reads every language style.** Character n-grams let a single model handle Sinhala and Tamil scripts, English, and their Latin-script forms without a translation step. Primary accuracy is at least 98.3% in every language group except `tanglish`.
3. **Urgency is almost error-free.** Of 95 urgent tickets, 94 were flagged and 1 was missed, with 0 false alarms on the 905 non-urgent tickets. The 0.30 cutoff deliberately favours catching urgent tickets over avoiding false alarms.
4. **Mistakes are scattered, not systematic.** No pair of primary categories is confused more than 1 time on the test fold. On the delivery-delay versus lost-property boundary we targeted, all 150 `delivery_delay` tickets were correct, and 1 of 50 `lost_item` tickets was predicted as `delivery_delay`.

Where it is weakest

5. **`lost_item` is the weakest primary class**, with F1 0.970 (recall 0.96, 50 test tickets).
6. **Rare secondary labels are less certain.** The lowest secondary F1 is `delivery_delay` at 0.963, from only 14 test tickets; secondary classes have between 8 and 30 examples each.
7. **`tanglish` is the weakest language group**: 86.4% accuracy and 77.8% macro-F1, but on only 22 tickets, so the estimate is uncertain.

What we learned about evaluation

8. **Internal splits of synthetic data can mislead model selection.** A first candidate chosen on an internal training split scored only 60.88% primary accuracy on the supplied validation set (an earlier baseline scored 62.88%), so it was rejected. The current configuration and cutoffs were then chosen using the supplied development validation set, and the reported scores come from a 25% fold of `train.csv` that the evaluated models were not fitted on.
9. **Very high scores call for caution, not celebration.** Near-perfect results on synthetic data can reflect repeated templates. The judges' unseen holdout and real customer messages are the true test.

Limitations

- The data are synthetic and language subgroups are uneven; the `mixed` and `tanglish` groups are too small for firm conclusions.
- Sinhala and Tamil text has not been reviewed by native speakers.
- Confidence is the model probability for the returned primary label; it is not independently calibrated.
- Human review stays essential for short, ambiguous, or low-confidence tickets.

Release record

- **Release version:** `v2.1.0` ; **model version:** `tf-phase2-drive-2.1.0` .
- **Training:** the deployed artifact is trained on 4,800 labelled rows; configurations and thresholds are frozen in `configs/training.json` .
- **Evaluation:** 3,000 / 1,000 stratified fit/test split from `train.csv` , seed 42; `validation.csv` excluded from test scoring.
- **Artifacts:** `artifacts/model.joblib` , `artifacts/evaluation_metrics.json` , `artifacts/evaluation_breakdown.json` , `artifacts/training_experiments.json` .
- **Reproduction:** run `python scripts/train_model.py` , or set `RETRAIN = True` in the setup cell, then run all cells.